

Sanskrit Document Retrieval-Augmented Generation (RAG) System

Technical Report

Submitted By: Swapnil Bagale

1. Introduction

This project implements a Retrieval-Augmented Generation (RAG) system for Sanskrit documents. The system is capable of ingesting Sanskrit text documents, retrieving relevant information based on user queries, and generating responses using a CPU-based Large Language Model (LLM).

The project demonstrates an end-to-end RAG pipeline operating entirely on CPU without requiring GPU acceleration.

The primary objective of this system is to:

- Process Sanskrit documents
 - Retrieve semantically relevant information
 - Generate meaningful responses
 - Provide an interactive question-answering interface
-

2. System Architecture

The system follows a standard Retrieval-Augmented Generation (RAG) architecture consisting of the following modules:

Components

1. Document Loader

Loads Sanskrit documents from:

- .txt
- .pdf

formats.

2. Preprocessing Module

Responsible for:

- Text cleaning
- Chunking
- Preparing data for embeddings

3. Embedding Model

Generates vector embeddings using multilingual sentence transformers.

4. Vector Database

Stores embeddings using FAISS for fast similarity search.

5. Retriever

Retrieves top relevant chunks for a user query.

6. Generator

Uses TinyLlama through Ollama to generate final responses.

7. User Interface

Gradio-based web interface for interaction.

3. Sanskrit Documents Used

The project used Sanskrit textual data stored in:

- .txt files
- Sanskrit PDF documents

Example Sanskrit content included:

- Bhagavad Gita verses
- Yoga-related Sanskrit text
- Sanskrit philosophical statements

Sample text:

योगः कर्मसु कौशलम्।
सत्यं वद। धर्मं चर।
विद्या ददाति विनयं।

4. Preprocessing Pipeline

The preprocessing pipeline includes the following steps:

4.1 Text Extraction

- PDFs were processed using PyMuPDF (fitz)
- TXT files were read directly

4.2 Text Cleaning

Basic cleaning operations:

- Removal of extra spaces
- Formatting normalization

4.3 Text Chunking

Documents were divided into smaller chunks using:

Chunk Size: 500 characters

Chunk Overlap: 50 characters

Chunking improves:

- Retrieval quality
- Embedding efficiency
- CPU performance

5. Embedding and Retrieval Mechanism

Embedding Model

The project used:

sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

Reason for selection:

- Supports multilingual text
- Better handling of Sanskrit/transliterated text
- Lightweight and CPU friendly

Vector Database

FAISS (Facebook AI Similarity Search) was used for vector storage and retrieval.

Benefits:

- Fast similarity search
- Efficient CPU execution
- Lightweight local indexing

Retrieval Process

For every user query:

1. Query embedding is generated
2. Similarity search is performed
3. Top 3 relevant chunks are retrieved

Code logic:

```
docs = db.similarity_search(query, k=3)
```

6. Generation Mechanism

The generator component uses:

TinyLlama 1.1B

through Ollama.

The retrieved context is combined with the user question and sent as a prompt to the language model.

Example prompt structure:

Context:

<retrieved chunks>

Question:

<user query>

Answer:

The model generates a response based on the retrieved Sanskrit context.

7. User Interface

The project includes a simple Gradio-based web interface.

Features:

- Text input for Sanskrit queries
- Response generation
- Browser-based interaction
- Lightweight local deployment

Run command:

```
python app.py
```

8. CPU Optimization Techniques

1. Lightweight LLM

TinyLlama 1.1B was selected because it:

- Runs efficiently on CPU
 - Requires less memory
 - Provides faster inference
-

2. Small Chunk Size

Using 500-character chunks reduced:

- Embedding latency
 - Retrieval time
 - Memory usage
-

3. FAISS Vector Search

FAISS provides:

- Fast local retrieval
 - Efficient nearest-neighbor search
 - CPU optimized performance
-

4. Top-K Retrieval

Only top 3 chunks were retrieved to:

- Reduce prompt size
 - Improve response speed
 - Lower inference cost
-

5. Quantized Ollama Models

Ollama’s optimized models improved:

- RAM efficiency
 - CPU inference speed
 - Local deployment performance
-

9. Performance Observations

Metric	Observation
Retrieval Speed	Fast
CPU Usage	Moderate
RAM Usage	Moderate
Response Time	3–8 seconds
GPU Requirement	None

The system successfully performs retrieval and generation tasks entirely on CPU.

10. Challenges Faced

Some challenges encountered:

- Limited Sanskrit-specific embedding models
- OCR/text extraction issues in some PDFs
- CPU-only inference constraints
- Context quality limitations with small LLMs

These were addressed using multilingual embeddings and lightweight models.

11. Conclusion

The project successfully demonstrates a functional Sanskrit Retrieval-Augmented Generation (RAG) system operating entirely on CPU.

The implemented pipeline:

- Processes Sanskrit documents
- Retrieves relevant contextual information
- Generates meaningful responses
- Provides an interactive interface

The system follows modular RAG architecture principles and can be extended further with:

- Better Sanskrit LLMs
 - Hybrid retrieval
 - Improved preprocessing
 - Larger datasets
-

12. Technologies Used

Technology	Purpose
Python	Core programming
LangChain	RAG framework
FAISS	Vector database

Technology	Purpose
Sentence Transformers	Embeddings
TinyLlama	Text generation
Ollama	Local LLM inference
Gradio	Web interface
PyMuPDF	PDF processing